

MoMo SMS REST API

Building and Securing a REST API

ALU | May 2026

Name

Nshimyumurwa Mary Therese
Aimable Bancunguye
Davy Dushimiyimana
Eloi Mizero
Clive Mushipe

Role

Team Lead / API Development
DSA & Data Parsing
Backend Support & Testing
Documentation & API Docs
Data Parsing & XML Processing

1. Introduction to API Security

APIs are the backbone of modern mobile-money systems. The MTN MoMo platform processes millions of transactions daily, and exposing this data through an API introduces real security risks if it's not handled properly. This report covers how we built and secured a REST API for MoMo SMS transaction data, and why the authentication method we used has limitations.

What is Basic Authentication?

HTTP Basic Auth works by encoding the username and password in Base64 and sending them in every request header. For example:

```
Authorization: Basic YWRtaW46bW9tbzIwMjQ=
```

That value is just Base64 of 'admin:momo2024'. It's not encryption at all — any tool can decode it in seconds. Without HTTPS, the credentials go over the network in plain text.

Why Basic Auth is weak

- No encryption by default. Base64 is encoding, not encryption. Anyone intercepting the request can read the credentials immediately.
- Credentials are sent on every single request. Unlike token-based auth, the actual password travels with every API call.
- No expiry. A stolen credential stays valid forever unless someone manually changes the password.
- No access levels. Everyone using the API gets the same permissions — you can't give one client read-only access and another write access.

Stronger alternatives

JWT (JSON Web Tokens)

The client logs in once and gets back a signed token that expires after a set time (e.g. 1 hour). After that, they send the token instead of their password. The token can also carry information like user roles, so the server knows what the client is allowed to do.

OAuth 2.0

This is the industry standard for this kind of thing. It supports different permission levels (scopes), token refresh, and is used by Google, GitHub, and MTN for their own APIs. It's what you'd use if third-party apps needed access to MoMo data on someone's behalf.

API Keys over HTTPS

A long random key per client, rotated every so often, sent only over TLS. It's a step up from Basic Auth and simpler to implement than OAuth for internal services.

2. Data Parsing — XML to JSON

The dataset `modified_sms_v2.xml` has 25 SMS records from the MoMo service. Each `<sms>` element has attributes for transaction type, amount, sender, receiver, date, and the original message body. We used Python's built-in `xml.etree.ElementTree` library so there are no extra dependencies to install.

The parsing function:

```
import xml.etree.ElementTree as ET

def parse_xml(filepath):
    transactions = []
    tree = ET.parse(filepath)
    root = tree.getroot()
    for sms in root.findall('sms'):
        transactions.append({
            'id': sms.get('id'),
            'transaction_type': sms.get('transaction_type'),
            'amount': float(sms.get('amount', 0)),
            'sender': sms.get('sender'),
            'receiver': sms.get('receiver'),
            'date': sms.get('date'),
            'currency': sms.get('currency', 'RWF'),
            'status': sms.get('status', 'completed'),
            'body': sms.get('body'),
        })
    return transactions
```

Sample output for record 1:

```
{
  "id": "1",
  "transaction_type": "incoming_money",
  "amount": 5000.0,
  "sender": "0781234567",
  "receiver": "0789876543",
  "date": "2024-01-03 08:12:00",
  "currency": "RWF",
  "status": "completed",
  "body": "You have received 5000 RWF from Alice..."
}
```

3. API Endpoints

Base URL: `http://localhost:8080` Auth: HTTP Basic (admin / momo2024)

GET /transactions

List all transactions

```
curl -u admin:momo2024 http://localhost:8080/transactions  
Response: {"count": 25, "transactions": [...]}
```

GET /transactions/{id}

Get one transaction by ID

```
curl -u admin:momo2024 http://localhost:8080/transactions/1  
Response: {"id": "1", "transaction_type": "incoming_money", "amount": 5000.0, ...}
```

POST /transactions

Create a new transaction

```
curl -u admin:momo2024 -X POST http://localhost:8080/transactions \\  
-H "Content-Type: application/json" \\  
-d '{"transaction_type": "transfer", "amount": 5000, ...}'  
Response: {"message": "Transaction created.", "transaction": {...}}
```

PUT /transactions/{id}

Update an existing transaction

```
curl -u admin:momo2024 -X PUT http://localhost:8080/transactions/1 \\  
-H "Content-Type: application/json" \\  
-d '{"status": "reviewed", "amount": 5500}'  
Response: {"message": "Transaction updated.", "transaction": {...}}
```

DELETE /transactions/{id}

Delete a transaction

```
curl -u admin:momo2024 -X DELETE http://localhost:8080/transactions/1  
Response: {"message": "Transaction '1' deleted successfully."}
```

Error codes

Code	Meaning	When it happens
200	OK	GET, PUT, DELETE succeeded
201	Created	POST succeeded
400	Bad Request	Missing fields or malformed JSON
401	Unauthorized	Wrong or missing credentials
404	Not Found	Transaction ID doesn't exist

4. DSA — Linear Search vs Dictionary Lookup

We compared two ways of finding a transaction by ID, running each one 100,000 times to get stable averages. The dataset has 25 records.

Method 1: Linear Search $O(n)$

Goes through the list one record at a time until it finds the right ID. In the worst case it has to check all 25 records.

Method 2: Dictionary Lookup $O(1)$

Stores all transactions in a Python dict keyed by ID. Python dicts are hash tables, so looking up a key takes constant time regardless of how many records there are.

Results (100,000 repetitions each):

Target ID	Scenario	Linear (us)	Dict (us)	Speedup
1	Best case (first)	0.075	0.048	1.6x
13	Middle (13/25)	0.300	0.050	6.0x
25	Worst case (last)	0.592	0.055	10.8x
99	Miss (not found)	0.593	0.049	12.2x

What the results show:

Linear search gets slower the further the target is in the list — 7.9x slower at the end compared to the beginning. When the ID doesn't exist it's just as slow because it has to scan every record before giving up.

Dictionary lookup stays basically flat across all cases (0.048 to 0.055 us). That's the whole point of a hash table — the lookup time doesn't depend on how many items are stored.

Other data structures worth knowing:

- **Binary Search Tree (BST):**

$O(\log n)$ search. Good when records are added/removed frequently in sorted order.

- **B-Tree:**

What databases actually use internally. Optimised for reading from disk on very large datasets.

- **Trie:**

Good for prefix searches, like finding all transactions from senders whose number starts with 078.

For this project, the dictionary is the right choice. $O(1)$ lookup, simple code, and the dataset fits easily in memory.

5. Conclusion

The API works end to end. All five CRUD operations are implemented and tested, Basic Authentication is enforced on every endpoint, and the server returns the right HTTP status codes in every case (200, 201, 400, 401, 404).

The DSA comparison confirmed what the theory predicts. Dictionary lookup is dramatically faster than linear search, especially for records near the end of the list or when an ID doesn't exist at all. The 12.2x speedup in the miss case makes the point clearly.

Basic Auth is fine for a coursework demo but it's not production-ready. The main problems are that credentials go out with every request and there's no expiry mechanism. For anything real, JWT would be the next step for internal clients and OAuth 2.0 for any third-party access, with everything over HTTPS.